

9.2 Строки, конкатенация и сравнение

00:00–00:04 - Интро

00:04–00:42 О чем вы узнаете в этом видео

Привет! Этот материал познакомит вас с понятием строки: вы узнаете, как создавать строки, что такое пустая строка, как измерять длину строки и как объединять две и больше строк в одну. Также вы узнаете, как сравнивать строки.

Работа со строками используется практически в любом программном обеспечении. Например, если вы пишете сообщение курьеру в службу доставки, то ваше сообщение и есть строка. Или если вы читаете описание товара в интернет-магазине, такое описание — тоже строка или даже множество строк, если это описание состоит из нескольких отдельных пунктов.

Давайте по порядку: что же такое строка?

00:43–03:20

Строка — в общем случае последовательность символов. Вы уже знаете, как создавать строки: пишете имя класса `string`, далее имя переменной, знак равно и в двойных кавычках строку, которая должна находиться в этой переменной, например:

```
String name = 'Дмитрий';
```

Строки также бывают пустыми, то есть не содержащими ни одного символа, и задать такую строку тоже можно. Выглядеть она будет так:

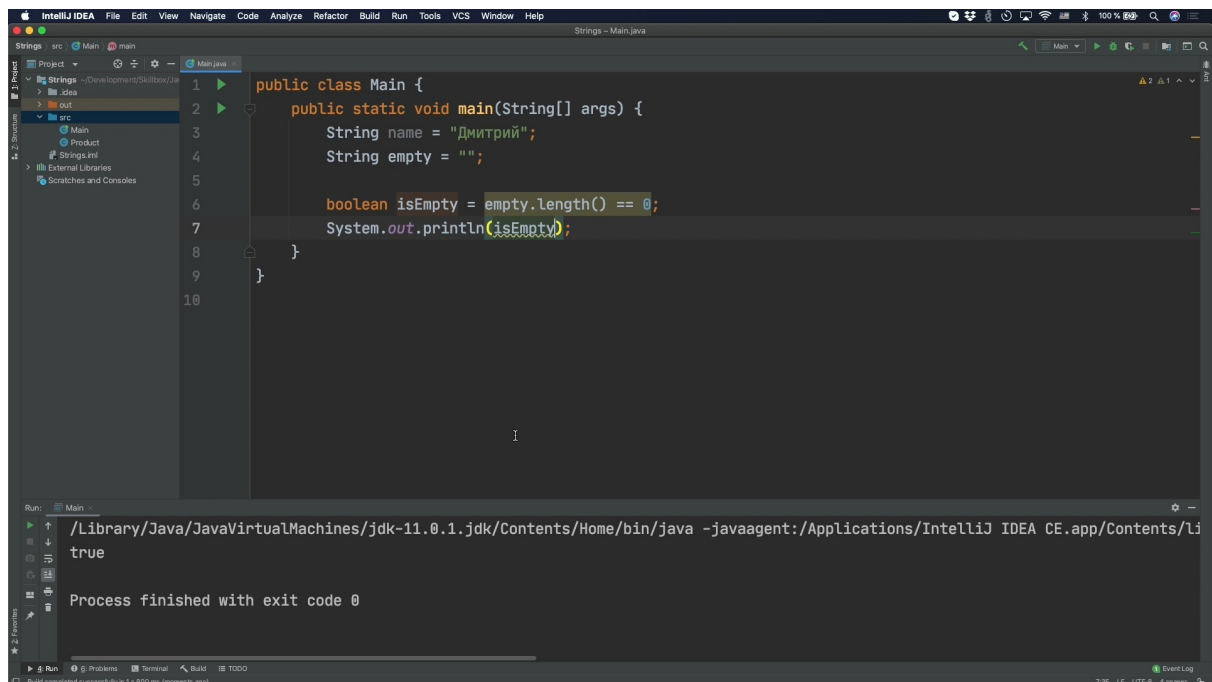
```
String empty = '';
```

Существует три способа, позволяющих узнать, пустая ли строка.

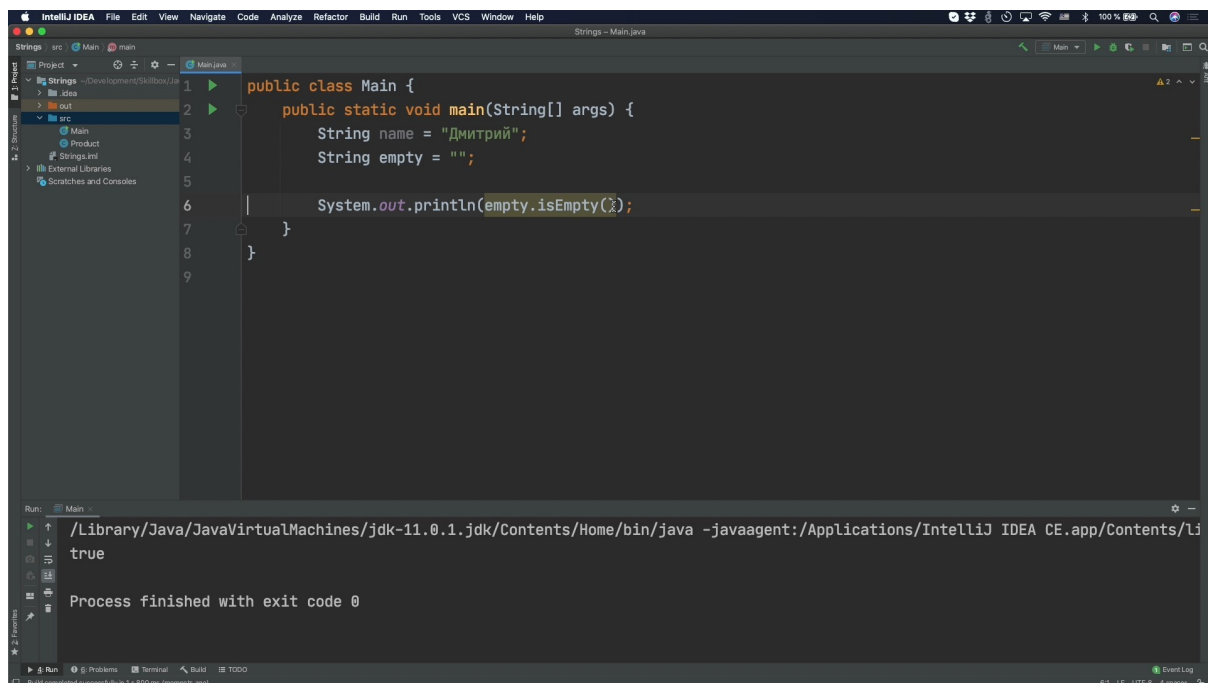
Первый способ — измерить её длину. Для этого создаём такую строку:

```
boolean isEmpty = empty.length() == 0;  
System.out.println(isEmpty)
```

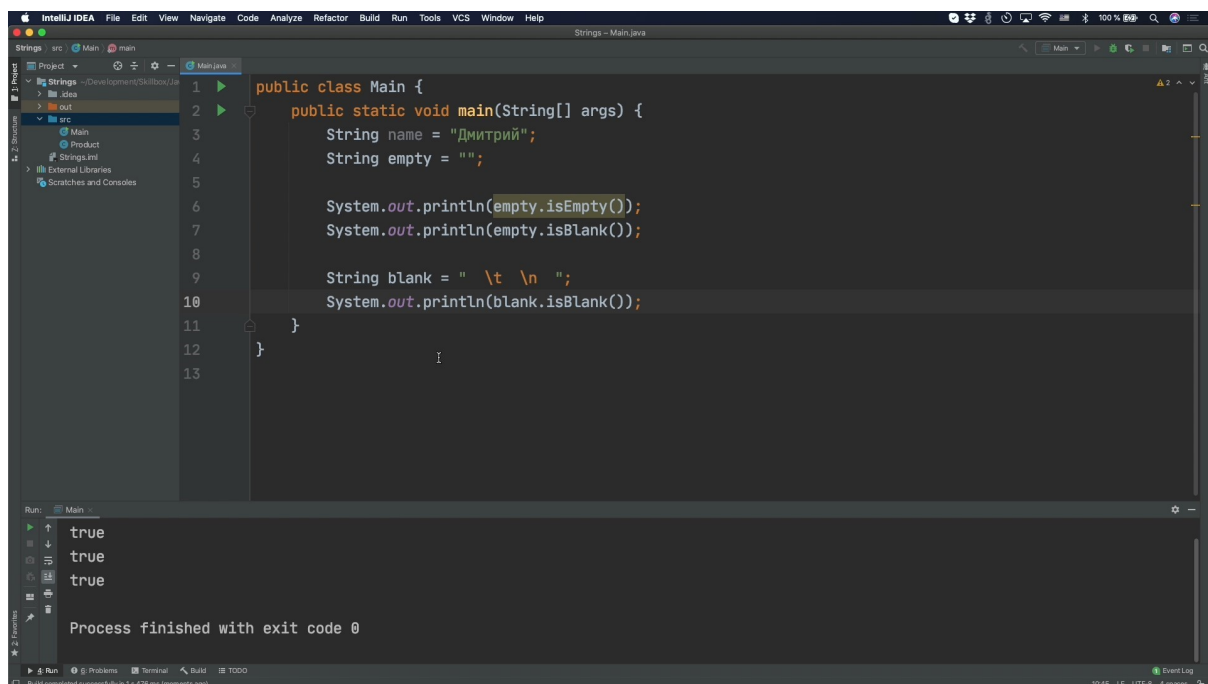
Если `boolean=true`, то строка пустая. Запускаем консоль и видим, что всё работает верно:



Но писать такое сравнение не всегда удобно, поэтому придумали специальный метод у класса `string` — `isEmpty`. Он также позволяет без проверки длины убедиться в том, что строка является пустой:



Но есть ещё один метод. Он возвращает true не только если строка пустая, но и если она состоит из пробелов или пробельных символов, например символов табуляции или переноса строки. Этот метод называется `isBlank` — вызовем этот метод у пустой строки, а также создадим пустую строку, которая будет состоять из пробелов и пробельных символов:



Как вы видите, при проверке мы получаем результат true.

Итак, есть три способа проверки пустой строки: проверка её длины, более удобный метод `isEmpty` и метод `isBlank`, возвращающий `true` не только когда строка пустая, но и когда она состоит из пробелов или пробельных символов.

Проверка: является ли строка пустой

```
String empty = "";
boolean isEmpty = empty.length() == 0;

System.out.println(empty.isEmpty());

String blank = " \t\n ";
System.out.println(blank.isBlank());
```

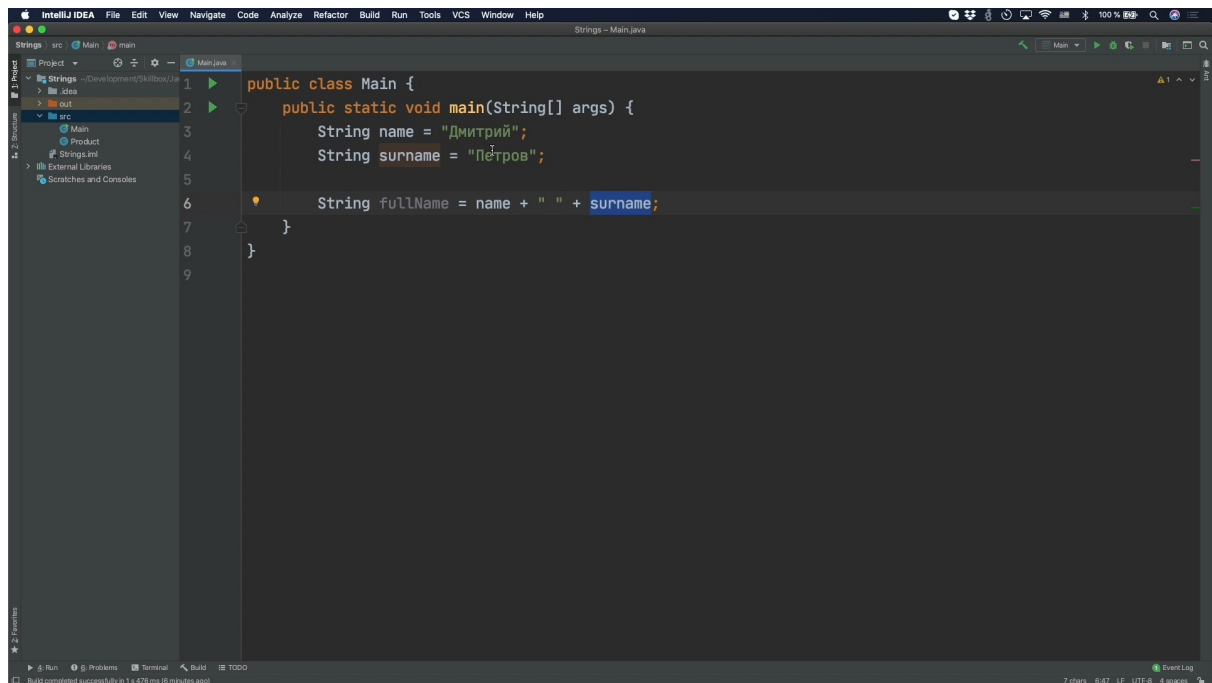
03:20–09:09 Объединение строк, или конкатенация

Давайте теперь подробно познакомимся с объединением строк. Объединение двух или большего количества строк в одну называется **конкатенацией**. Давайте сразу посмотрим на практике несколько примеров конкатенации.

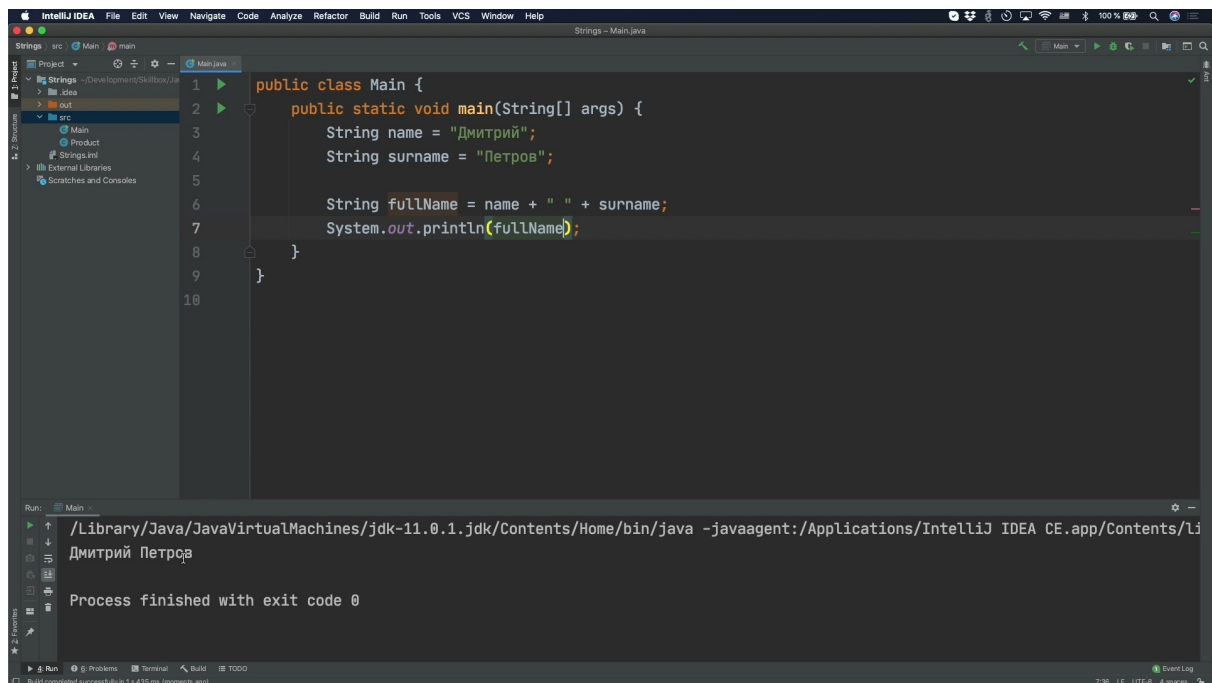
Первый пример простой: у нас есть две переменные. В первой хранится имя человека, а во второй — его фамилия. Нужно создать строку, в которой будет и имя, и фамилия через пробел.

Для этого создадим переменную, назовём её `full name` — полное имя — и напишем `name`, то есть имя человека, плюс в кавычках, пробел, + `surname` — фамилия.

Таким образом, мы объединим в строку `full name` сразу три строки:



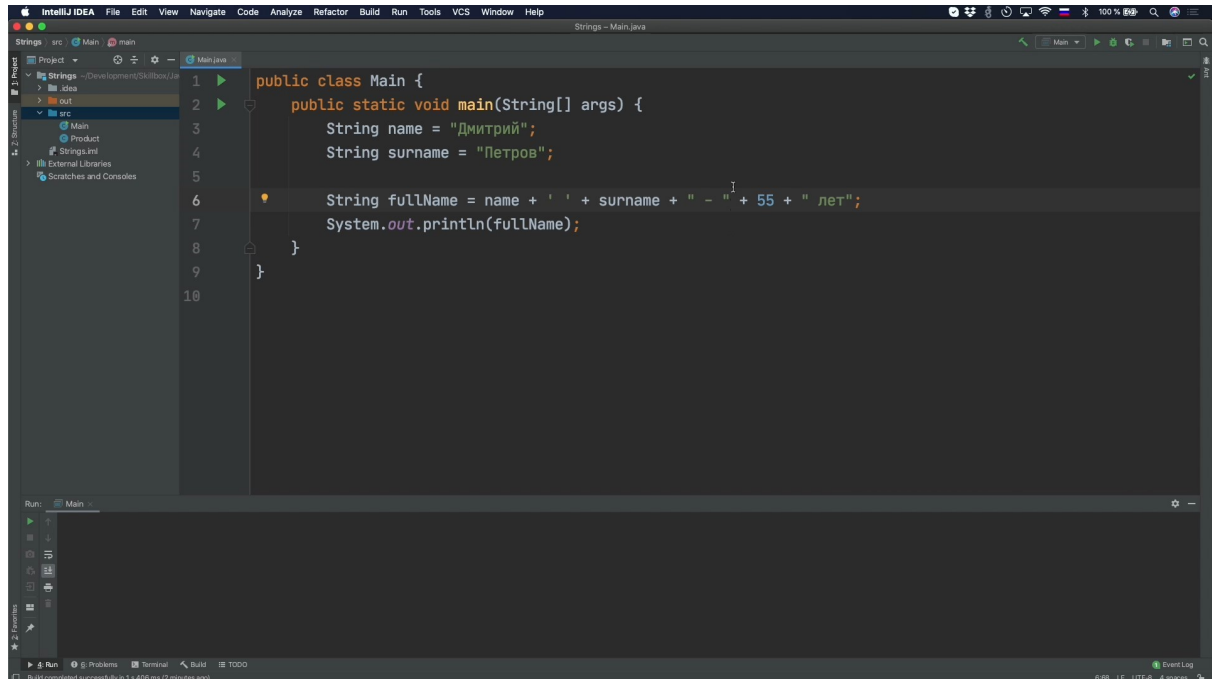
Получаем в итоге полное имя человека:



То есть конкатенация строк производится с помощью знака плюс. Объединяя несколько строк таким образом, вместо строк вы можете также использовать и другие объекты или даже примитивы.

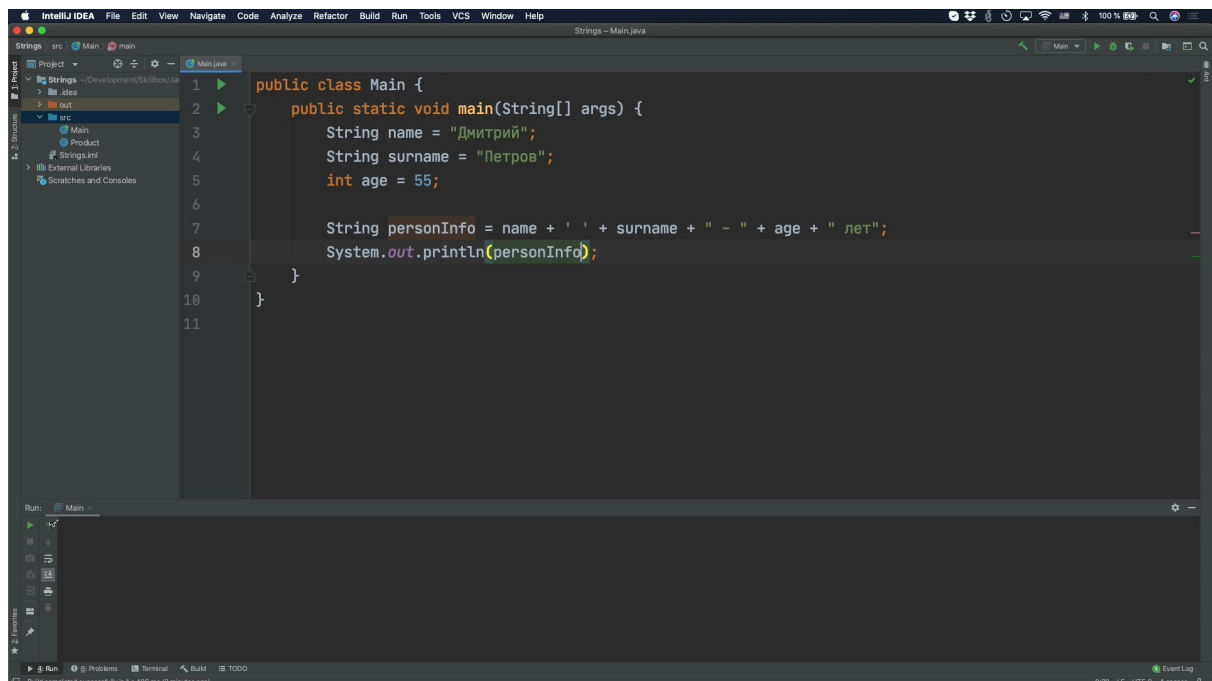
Вместо строки с пробелом вы можете использовать символ «пробел» в одинарных кавычках или даже использовать числа, например в

строке full name можно дописать возраст этого человека, поставить дефис, допустим: 55, + лет.



```
1 public class Main {
2     public static void main(String[] args) {
3         String name = "Дмитрий";
4         String surname = "Петров";
5
6         String fullName = name + ' ' + surname + " - " + 55 + " лет";
7         System.out.println(fullName);
8     }
9 }
10
```

Это число можно также вынести в переменную:



```
1 public class Main {
2     public static void main(String[] args) {
3         String name = "Дмитрий";
4         String surname = "Петров";
5         int age = 55;
6
7         String personInfo = name + ' ' + surname + " - " + age + " лет";
8         System.out.println(personInfo);
9     }
10 }
11
```

Поскольку мы добавили информацию, которая не является именем, переменная будет называться уже не full name, а person info.

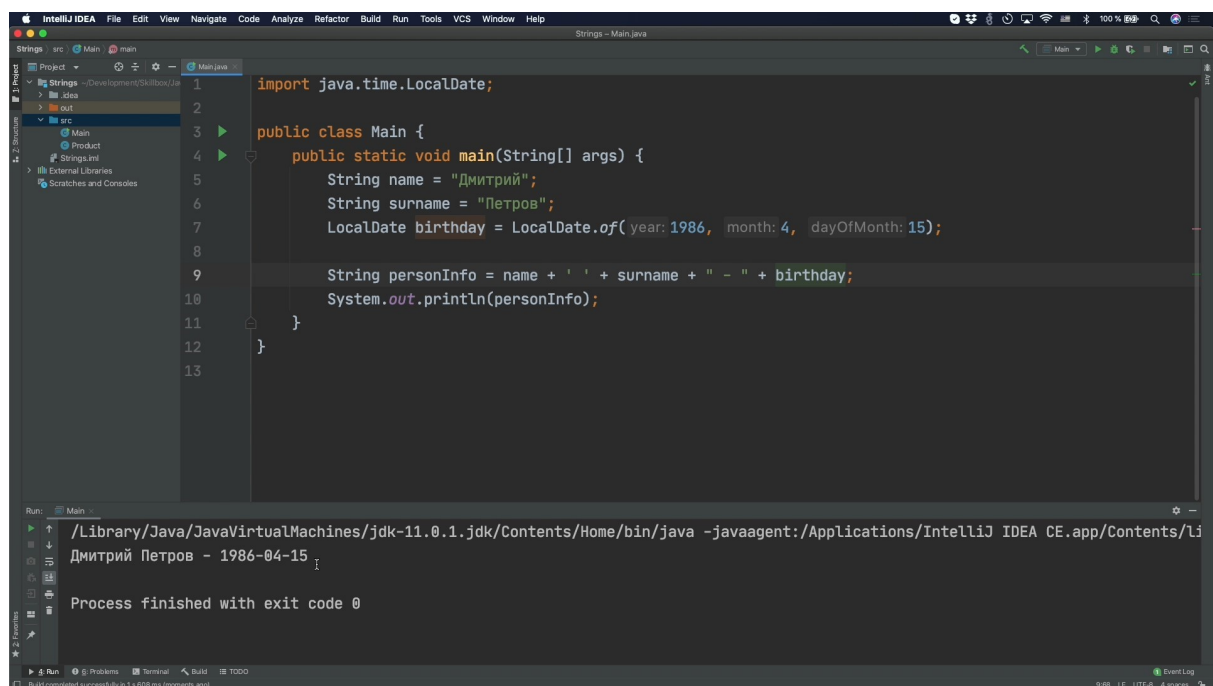
В данном случае при конкатенации будет происходить автоматическое преобразование примитивов символа и числа в строки. А что будет, если со строкой сконкатенировать объект?

Например, у нас вместо возраста есть дата рождения:

```
Local Date birthday = Local.Date.of( year: 1986, month: 4,
dayOfMonth:15 );
```

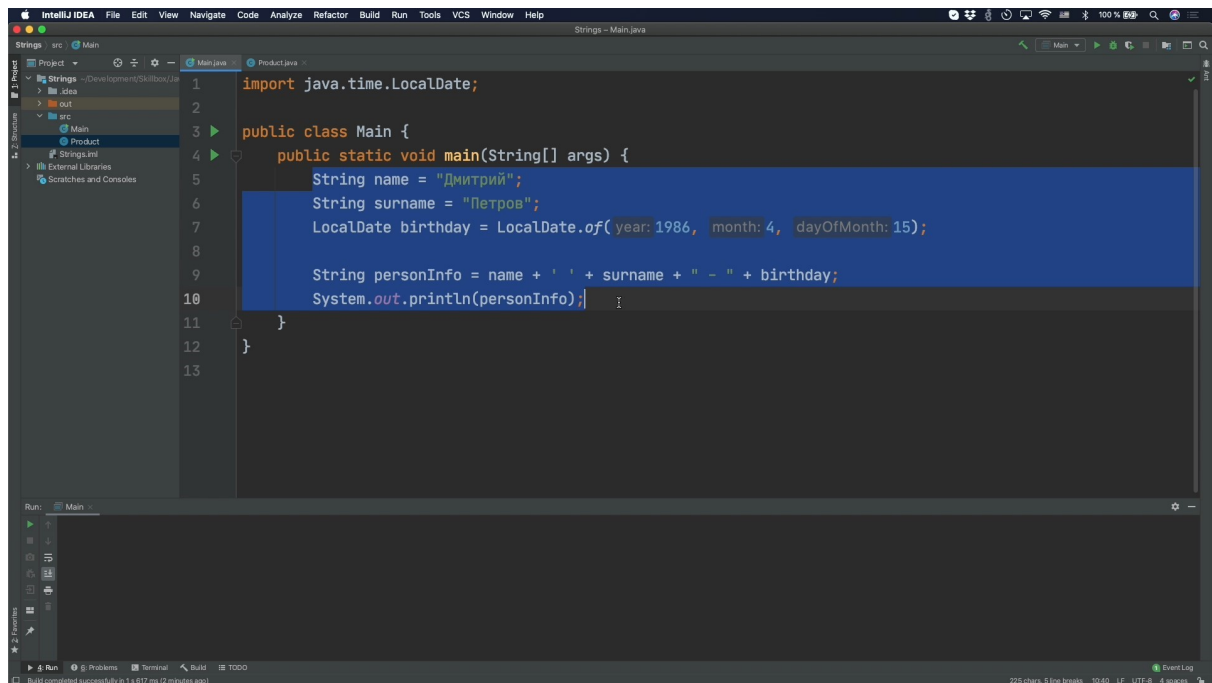
После добавления такой строки можно убрать из строки personal info число и слово «лет» и вместо них вставить слово birthday.

Результат:

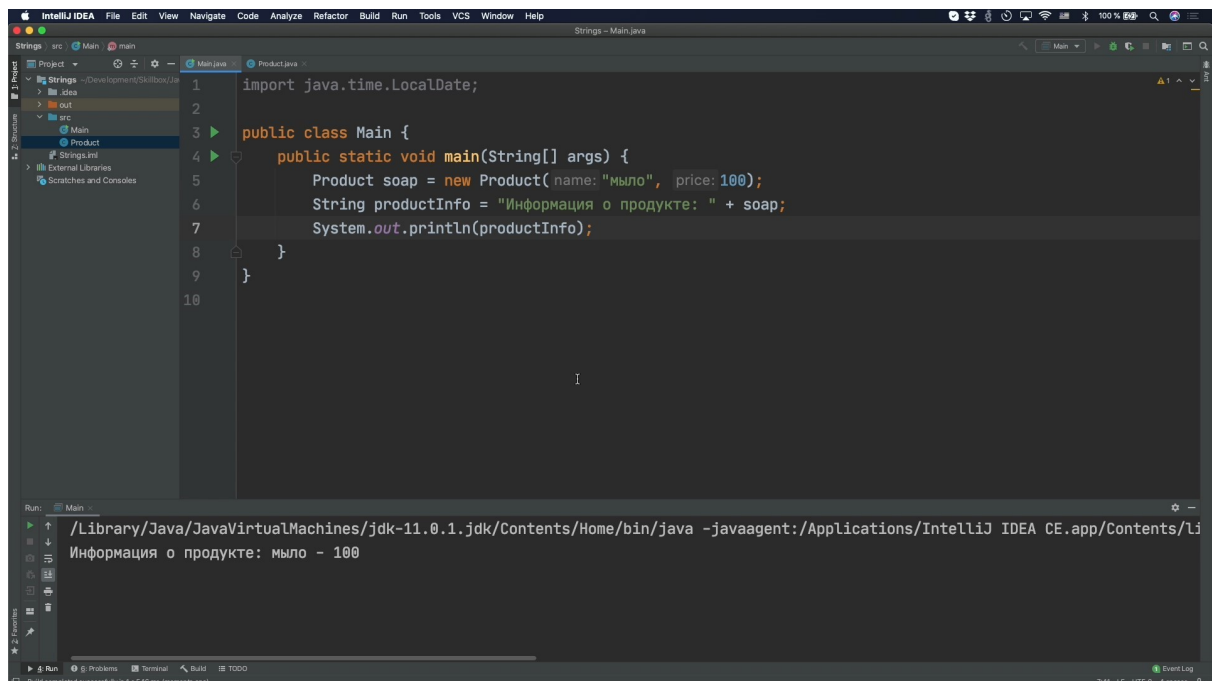
The screenshot shows the IntelliJ IDEA IDE. The main editor window displays a Java file named 'Main.java'. The code includes an import for 'java.time.LocalDate', a 'Main' class with a 'main' method. Inside the 'main' method, variables 'name' and 'surname' are set to 'Дмитрий' and 'Петров' respectively. A 'LocalDate' object 'birthday' is created using 'LocalDate.of' with year 1986, month 4, and dayOfMonth 15. These are concatenated into a 'personInfo' string, which is then printed using 'System.out.println'. The left sidebar shows the project structure with 'Main.java' selected. The bottom panel shows the 'Run' configuration and the output of the program: 'Дмитрий Петров - 1986-04-15'. The status bar at the bottom indicates 'Build completed successfully in 1 s 658 ms (moments ago)'.

Что произошло? При конкатенации со строками у объектов автоматически вызывается метод toString — этот метод есть у всех объектов Java, — и если в нём в явном виде прописано, как нужно выводить информацию об этом объекте, то преобразование такого объекта строк будет понятно.

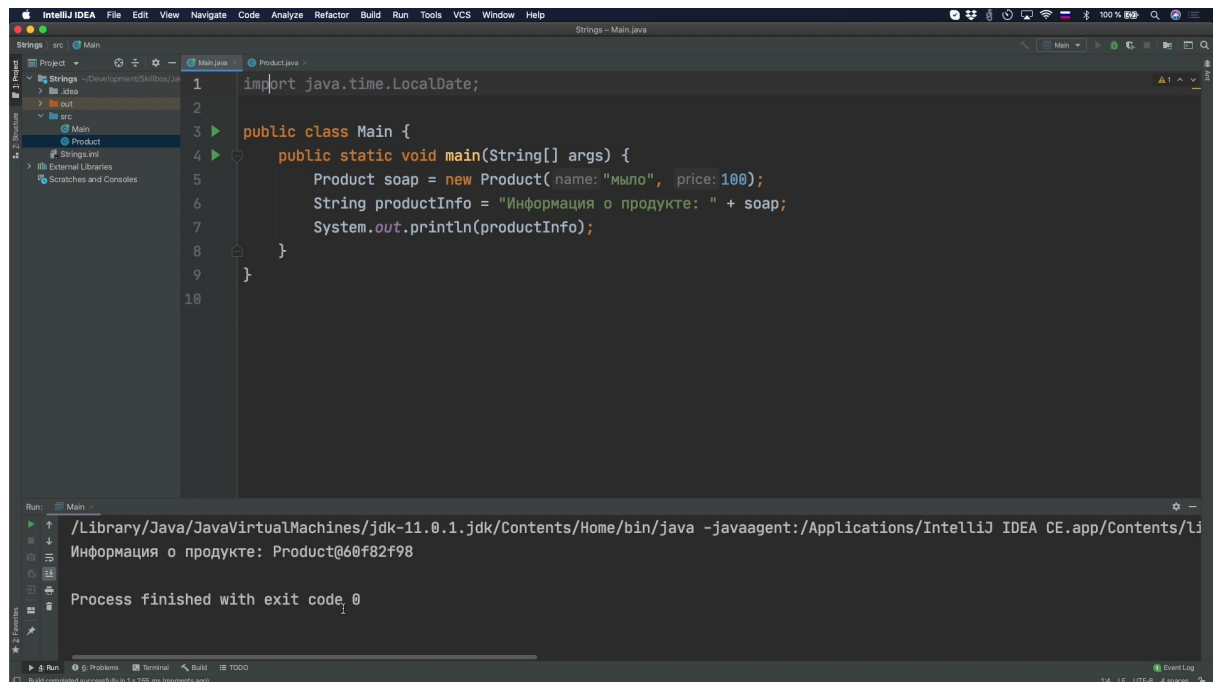
Давайте посмотрим ещё на одном примере. У нас есть класс product, объект этого класса — некие продукты, у которых есть название и цена. У класса тоже есть метод toString, в котором написано, что он возвращает имя продукта — дефис — пробел — дефис и цену продукта соответственно. Если мы напишем вот такой код:



и создадим продукт, например мыло стоимостью 100 рублей условно, и дальше создадим строку product info, в которую сконкатенируем другую строку и объект soap — объект класса product, то мы получим полноценную информацию о продукте, которая возвращается методом toString класса products, который, в свою очередь, вызывается при конкатенации объекта этого класса со строкой.



Если бы у класса product не было метода toString, то вывод был бы не таким красивым. Выглядел бы он так:



The screenshot shows the IntelliJ IDEA IDE. The main editor displays a Java file named `Main.java` with the following code:

```
1 import java.time.LocalDate;
2
3 public class Main {
4     public static void main(String[] args) {
5         Product soap = new Product(name: "мыло", price: 100);
6         String productInfo = "Информация о продукте: " + soap;
7         System.out.println(productInfo);
8     }
9 }
10
```

The left sidebar shows the project structure with folders `src` and `out`, and files `Main.java` and `Product.java`. The bottom panel shows the Run output:

```
Run: Main
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li
Информация о продукте: Product@60f82f98
Process finished with exit code 0
```

Кстати, конструкция `System.out.println`, которую мы постоянно используем для вывода в консоль, тоже автоматически вызывает у всех объектов метод `toString`.

Важно запомнить, что метод `toString` есть у всех объектов Java.

Даже если вы создаёте свой класс и не задаёте ему метод `toString`, то он всё равно будет, только он не будет выводить содержимое этого объекта. Если вам нужно, чтобы объекты вашего класса преобразовались в понятные строки, вам нужно реализовать метод `toString` и прописать в соответствующий код.

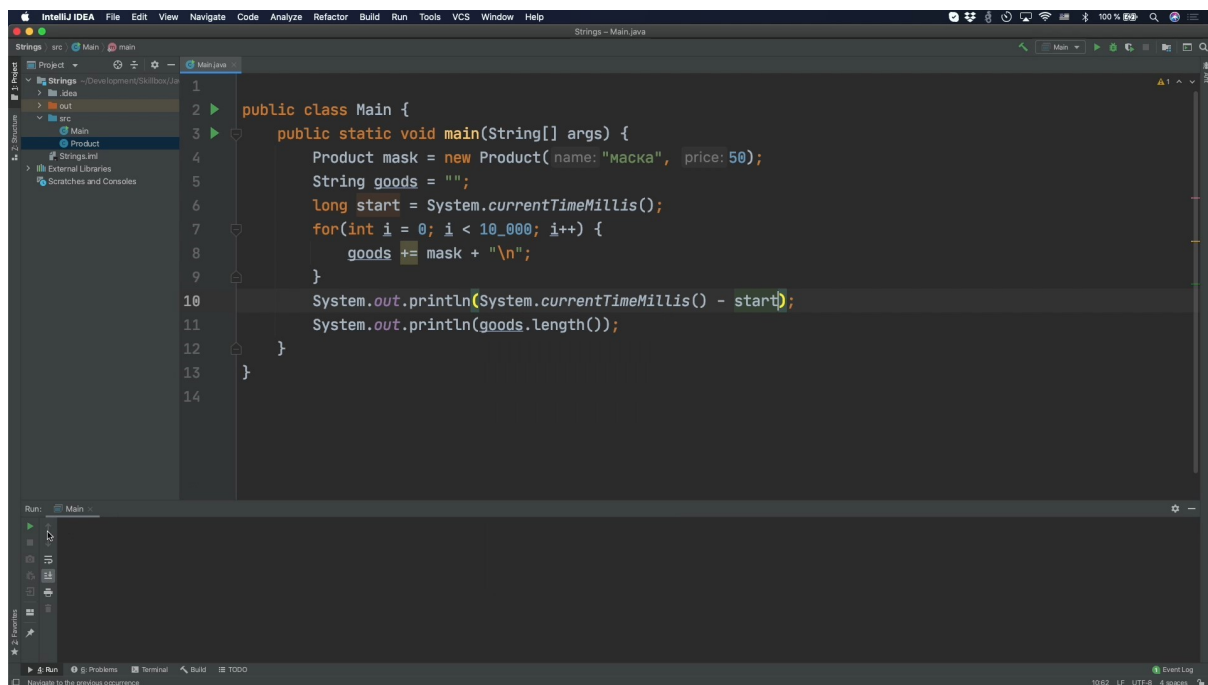
Таким образом, метод `toString`, предназначенный для преобразования объектов в строки, вызывается у этих объектов автоматически при конкатенации со строками или при передаче такого объекта в качестве параметров конструкции `System.out.println`.

09:09–13:07 Двойная конкатенация

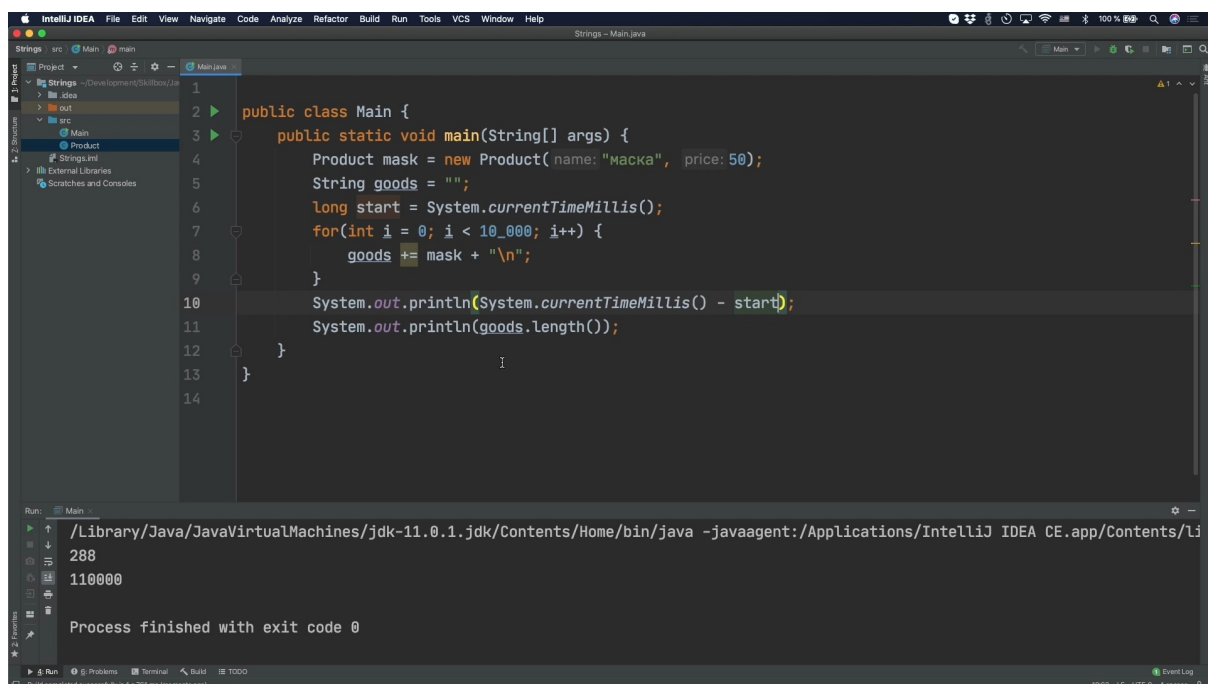
Теперь поговорим о множественной конкатенации, когда вам к строке нужно добавить множество новых строк. Здесь есть одна опасность, которую нужно понимать и о которой нужно помнить.

Например, вам нужно вывести в одну строку все товары из интернетмагазина. Пусть у нас будет один товар, например маска стоимостью 50 рублей, и мы выведем её 10 тысяч раз. Для этого создадим строку, в которую мы будем выводить этот товар, `string goods` — в переводе с английского «товары». Далее напишем цикл `for` от 0 до 10 000. После этого напишем строку `«goods +=»` — здесь написан оператор «плюс равно» — это конкатенация с присвоением, то есть в конец этой строки будет добавлена та строка, которую напишем справа от этого оператора. Справа мы, собственно, напишем `mask`, плюс и перенос строки.

В конце выведем общую длину получившейся строки — посмотрим, как долго будет выполняться такой код. Создадим переменную `start` с текущей меткой времени и далее выведем сразу после цикла разницу между текущими активами и переменной.



```
1 public class Main {
2     public static void main(String[] args) {
3         Product mask = new Product(name: "Macka", price: 50);
4         String goods = "";
5         long start = System.currentTimeMillis();
6         for(int i = 0; i < 10_000; i++) {
7             goods += mask + "\n";
8         }
9         System.out.println(System.currentTimeMillis() - start);
10        System.out.println(goods.length());
11    }
12 }
13
14
```

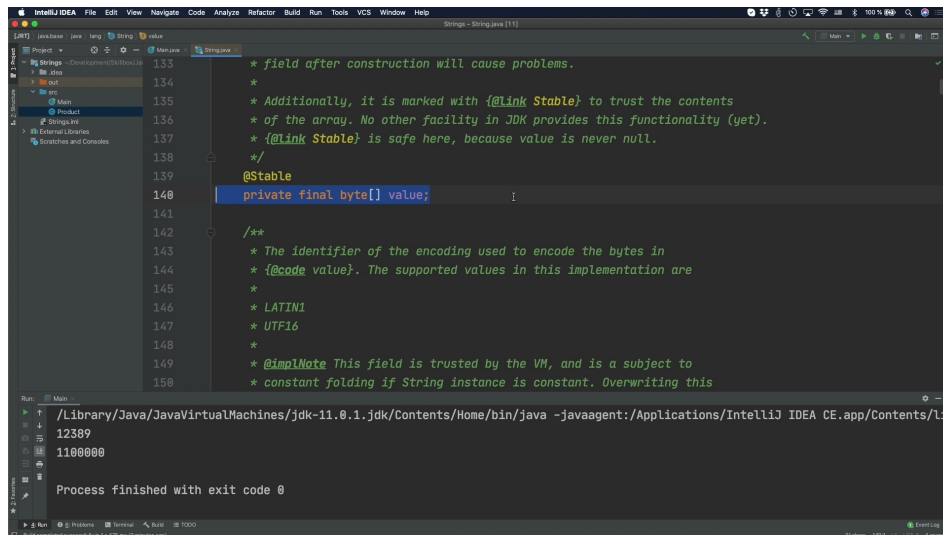


```
Run: Main
288
110000
Process finished with exit code 0
```

Запускаем код и видим, что такой код выполнялся бы 288 миллисекунд. Если мы увеличим число до 100 тысяч, то время так же увеличится, но гораздо больше чем в десять раз — код в 100 тысяч повторений выполнялся бы 12 389 миллисекунд.

В этом моменте и прячется опасность. При множественной конкатенации ваша программа может начать работать гораздо медленнее. Давайте поймём, почему так происходит.

Чтобы это понять, нужно перейти в исходный код класса `String`. Чтобы перейти в исходный код класса `String`, нужно нажать `Ctrl` или `Command` и на название этого класса. Мы видим такую строку:



Это, по сути, место, в котором хранится строка. Обратите внимание, что она хранится в виде массива байтов — квадратные скобки как раз означают, что эта переменная является массивом. Массив — множество фиксированного размера, то есть в данном случае множество байтов фиксированного размера.

О массивах мы ещё поговорим в одном из следующих модулей. Здесь, скорее, важно, что эта переменная обозначена ключевым словом `final`, то есть она не меняется после создания объекта. Это приводит к тому, что строки становятся иммутабельными.

То есть если вы конкатенируете одну строку с другой, то в памяти формируется новый объект строки, длина которой равна суммарной длине первой и второй строк. После чего в новую строку переписывается каждый символ из обеих строк, которые мы конкатенируем, и чем больше процессов создания новых объектов строк, тем большее время занимает конкатенация строки в цикле. Поскольку при каждом изменении строки создаётся новый объект, по итогу выполнения этого кода мы создадим 100 тысяч объектов. Чем больше объектов строк мы создаём, тем дольше выполняется этот

код. Кроме того, итоговая строка — в нашем коде это переменная `goods` — становится всё длиннее и длиннее, и добавление к ней даже маленькой строки проходит всё дольше и дольше.

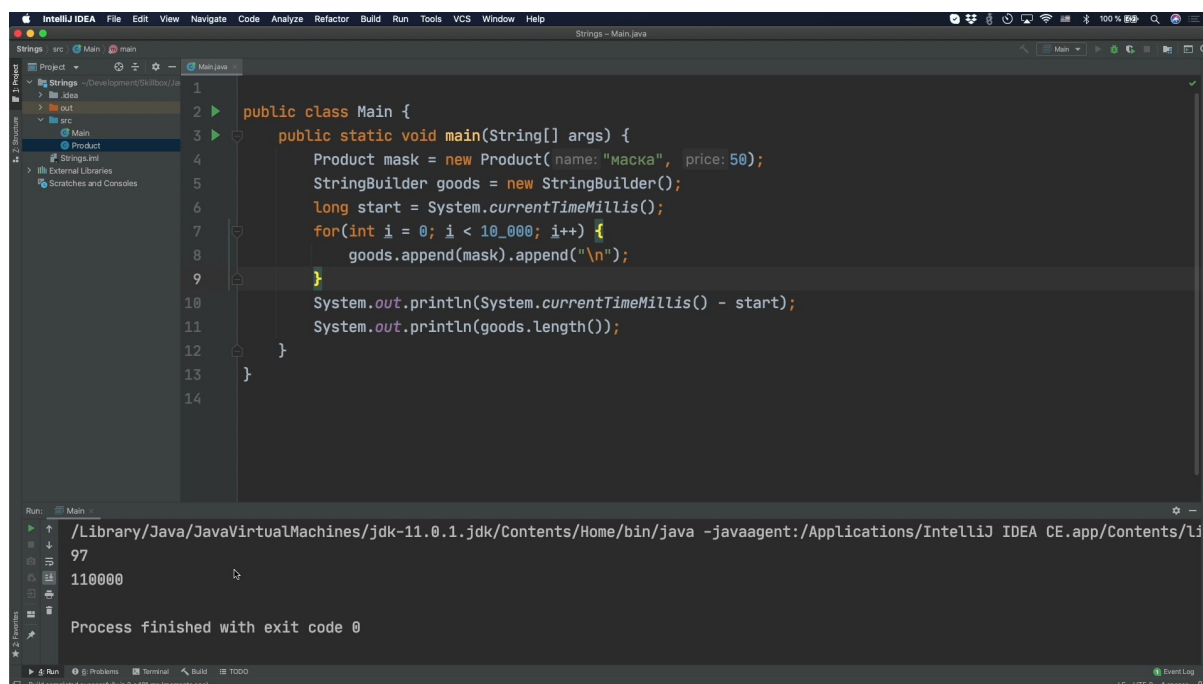
13:07–17:03

Как выйти из этой ситуации? Есть два способа: классический и современный.

Классический способ — это использование специального класса `StringBuilder`, в котором происходит добавление новой строки за одно и то же время, независимо от её длины. Давайте убедимся в эффективности этого класса на практике.

`Stream` переименуем в `StringBuilder`, оставим название `goods`, только вместо равно `=` и пустых кавычек напишем `new StringBuilder`. И далее будем использовать метод `append` — `append("mask")` и `append("\n")`.

Далее весь код оставим таким же, каким он был. Вспомним, что этот код выполнялся в течение 12 секунд. Давайте запустим и посмотрим новое время выполнения — оно составило 97 миллисекунд.



```
1 public class Main {
2     public static void main(String[] args) {
3         Product mask = new Product(name: "маска", price: 50);
4         StringBuilder goods = new StringBuilder();
5         long start = System.currentTimeMillis();
6         for(int i = 0; i < 10_000; i++) {
7             goods.append(mask).append("\n");
8         }
9         System.out.println(System.currentTimeMillis() - start);
10        System.out.println(goods.length());
11    }
12 }
13
14
```

Run: Main

```
/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Li
97
110000

Process finished with exit code 0
```

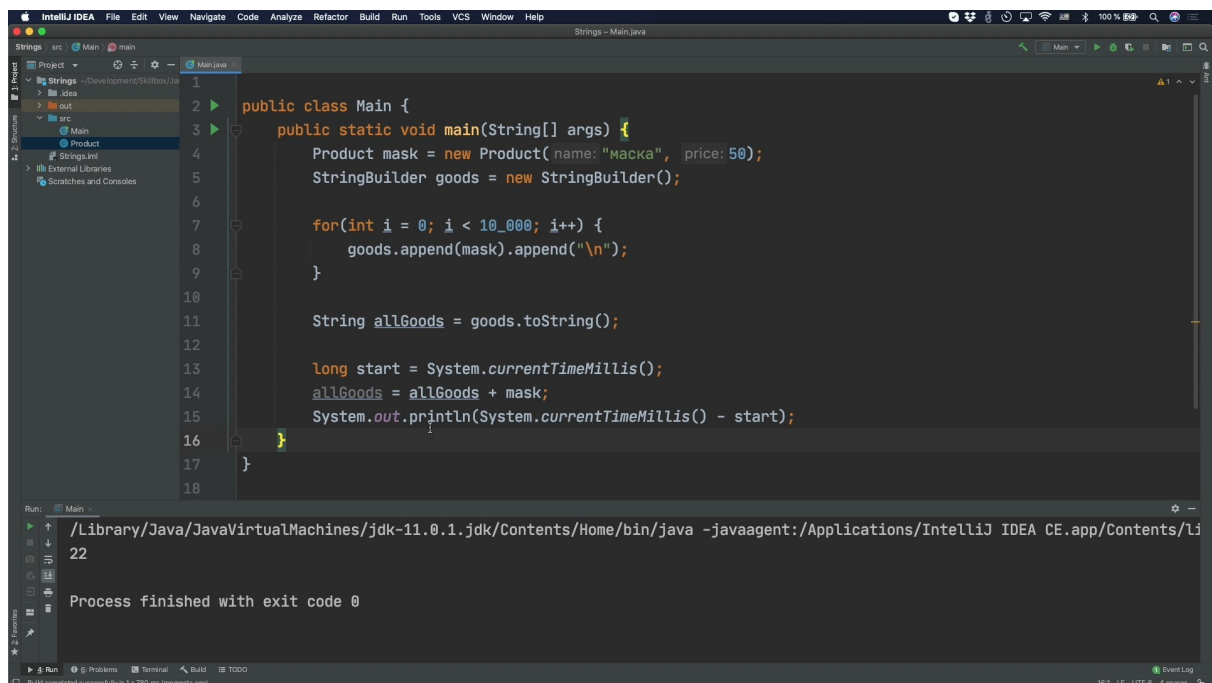
Есть и другой способ. У класса `String` есть метод `concat`, который работает гораздо быстрее, чем оператор «`+=`». Давайте в этом убедимся на практике. Создадим новые строки:

```
String allGoods = goods.toString(); allGoods
= allGoods + mask;
```

`allGoods` означает «все товары». Перенесём строку `long start`:

```
String allGoods = goods.toString(); long
start = System.currentTimeMillis(); allGoods
= allGoods + mask;
```

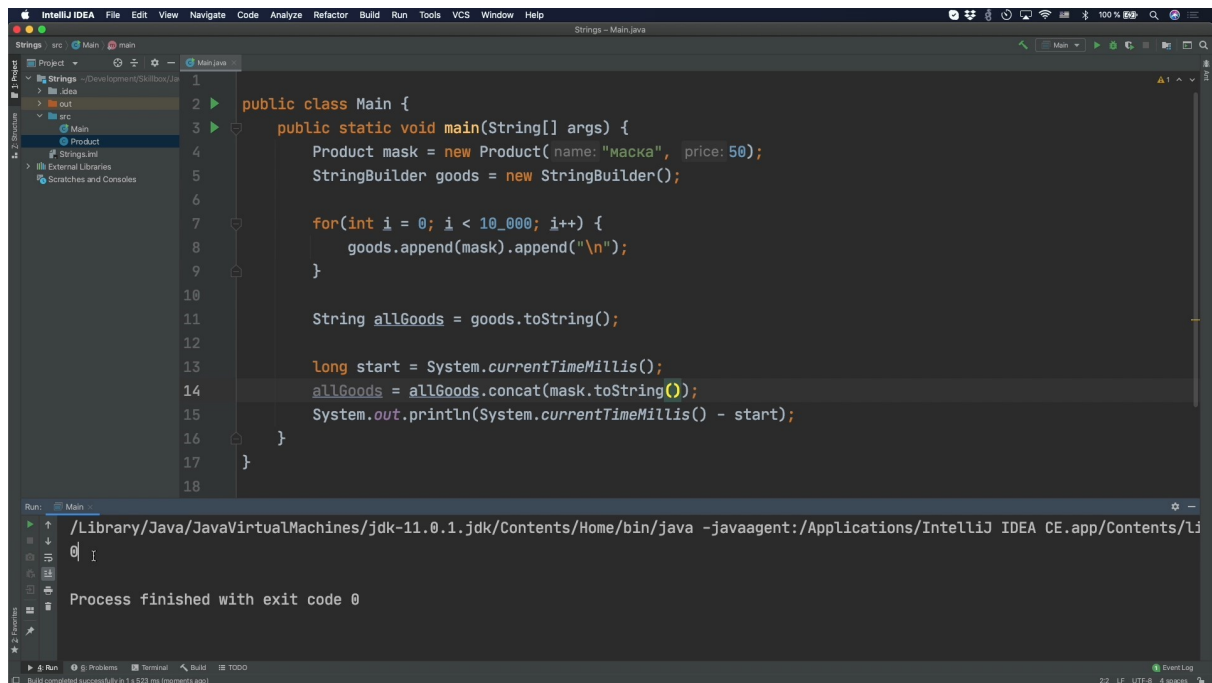
Посмотрим длительность выполнения этой операции:

The screenshot shows the IntelliJ IDEA IDE. The main editor displays a Java file named 'Main.java' with the following code:

```
1 public class Main {
2     public static void main(String[] args) {
3         Product mask = new Product(name: "маска", price: 50);
4         StringBuilder goods = new StringBuilder();
5
6         for(int i = 0; i < 10_000; i++) {
7             goods.append(mask).append("\n");
8         }
9
10        String allGoods = goods.toString();
11
12        long start = System.currentTimeMillis();
13        allGoods = allGoods + mask;
14        System.out.println(System.currentTimeMillis() - start);
15    }
16 }
17
18
```

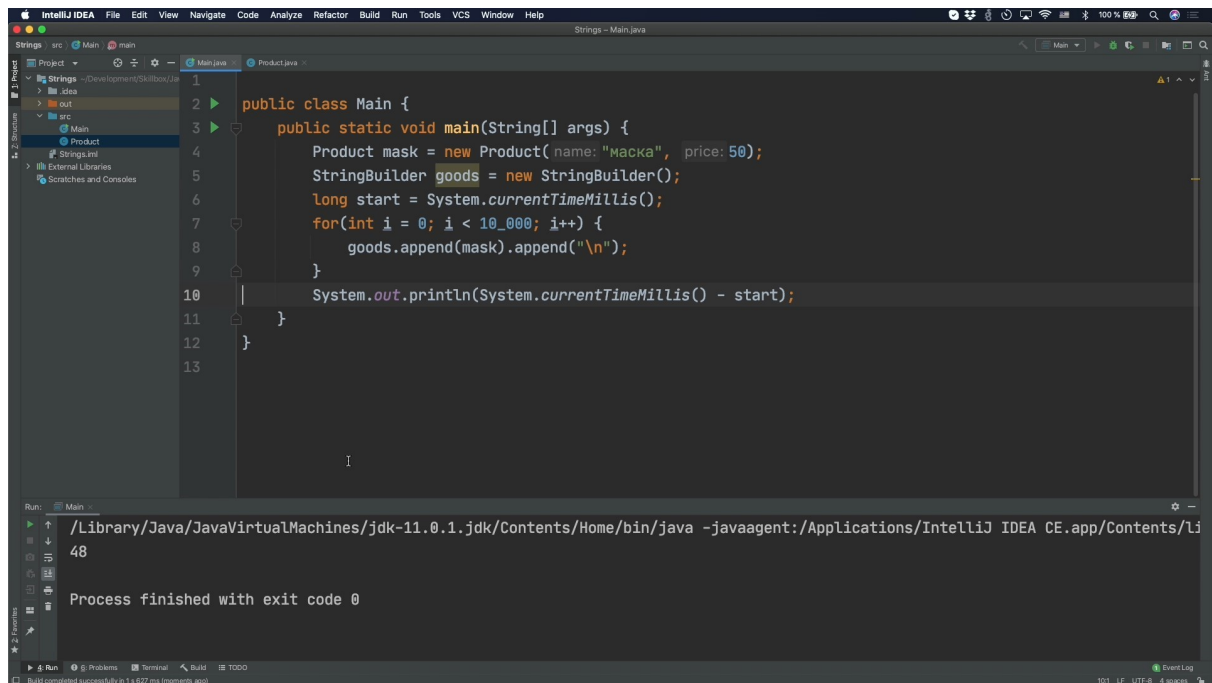
The 'Run' tab at the bottom shows the command executed: `/Library/Java/JavaVirtualMachines/jdk-11.0.1.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ IDEA CE.app/Contents/Lj22`. Below the command, it states 'Process finished with exit code 0'. The status bar at the bottom indicates 'Build completed successfully in 1 s 780 ms (moments ago)'.

Длительность такой операции составила 22 миллисекунды. Если добавить сюда метод `concat`, то длительность такой операции будет 0 миллисекунд:

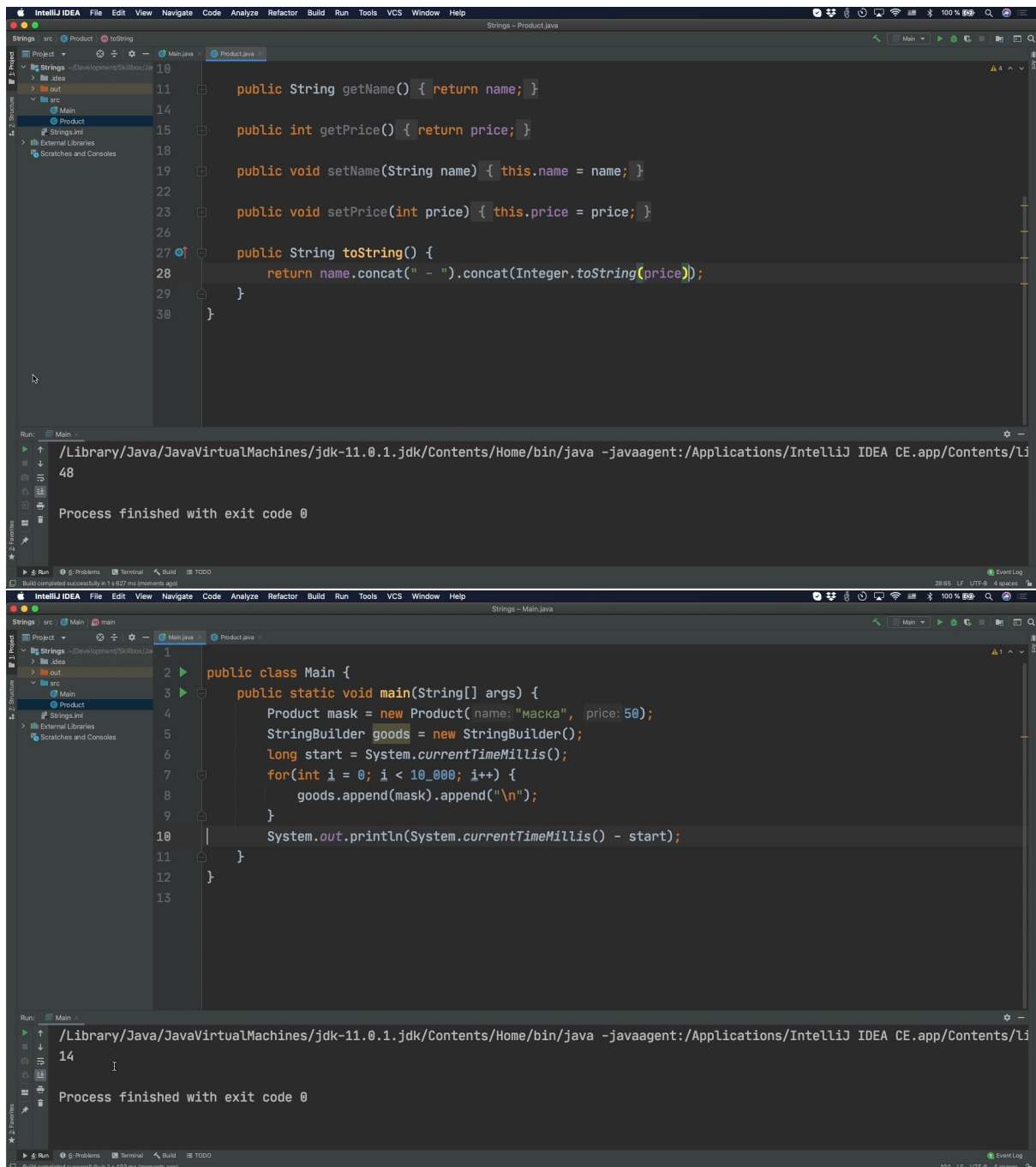


То есть в целях оптимизации кода для конкатенации строк, особенно если эта операция постоянно повторяется, лучше использовать метод `concat`. Вот, например, в нашем же коде, где мы формируем строку, у нас используется объект класса `product`, в котором конкатенация происходит с помощью оператора «+». Давайте заменим этот оператор на метод `concat` и посмотрим, насколько ускорится код.

Для этого сначала измерим длительность выполнения нашего кода. Вернём всё в исходное состояние и проверим — этот код выполняется за 48 миллисекунд:



Далее в классе product вместо + напишем concat и ещё раз concat. В данном случае price — это число int, примитив. Превратим его в строку integer и запустим этот код снова.



Теперь такой код занимает 14 миллисекунд.

Давайте подытожим эту тему. Для конкатенации строк можно использовать три основных способа:

- Оператор «+».
- Оператор `StringBuilder` — в случае если у вас множество повторяющихся операций конкатенации.
- Метод `concat` класса `string`.

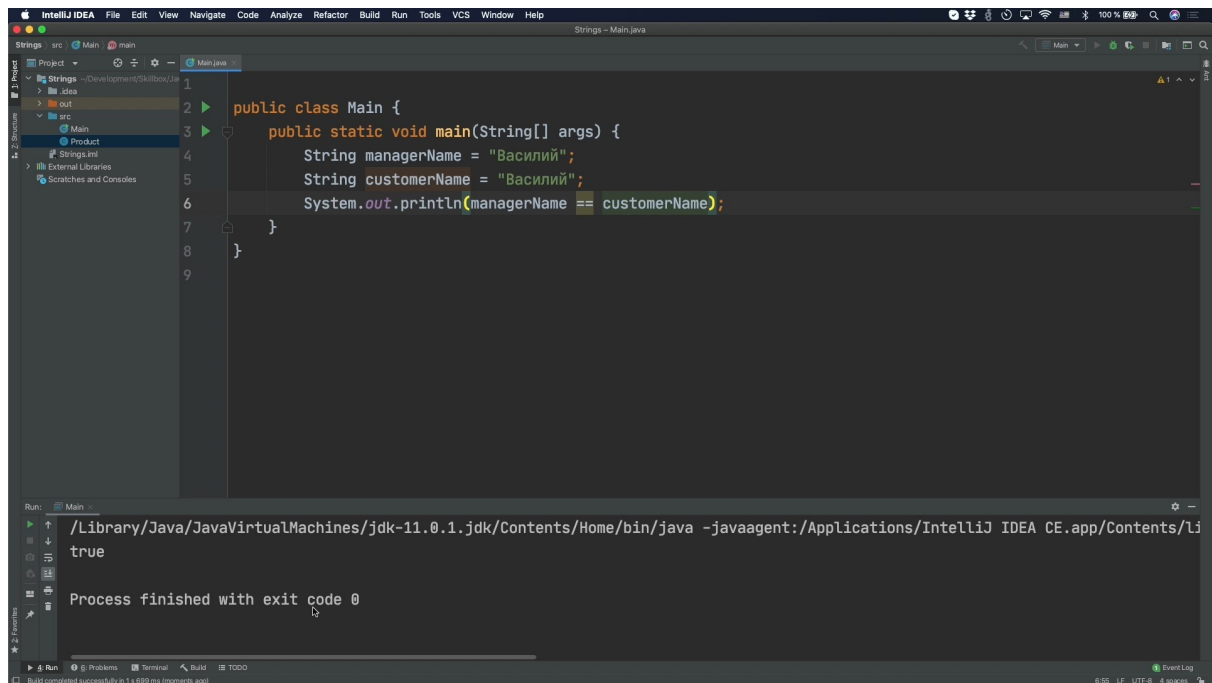
Конкатенация строк

```
String fullName = name +  
    " " + surname;  
  
StringBuilder text =  
    new StringBuilder();  
text.append(word);  
  
String personInfo = fullName  
    .concat(" - ")  
    .concat(birthday);
```

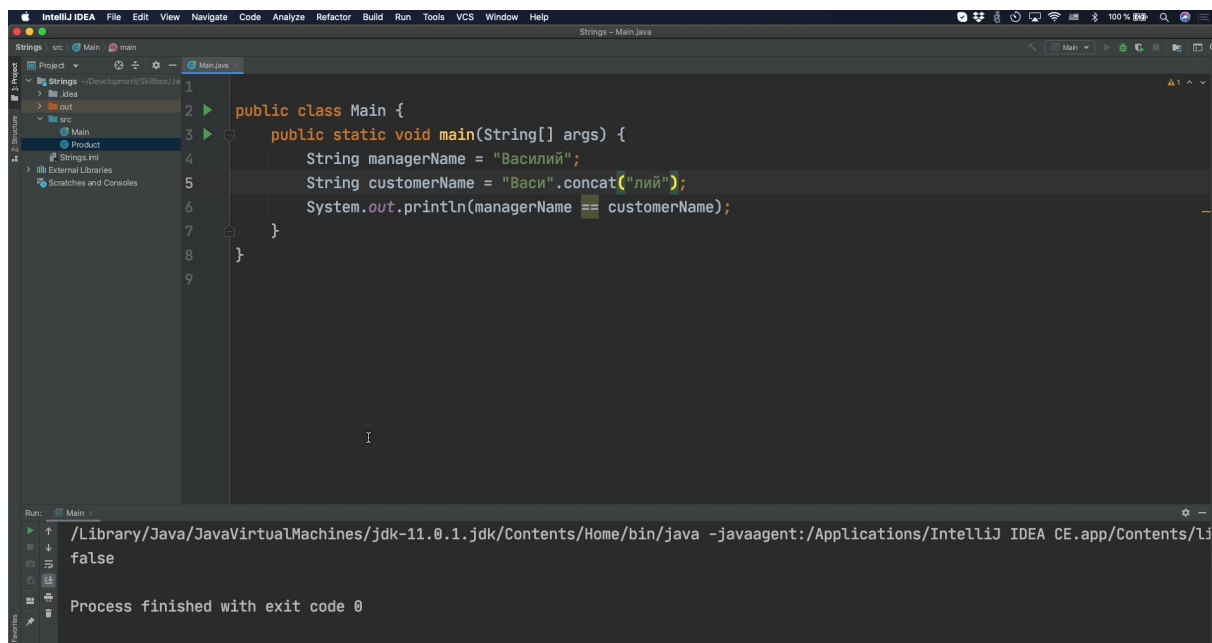
17:03–18:58 Сравнение строк

Следующая тема, которую нам необходимо затронуть, — это сравнение строк. Давайте сразу на практическом примере посмотрим, как это делается.

Вот у нас есть две одинаковые строки, и если сравнить их с помощью двойного равно, то мы получим результат true. Давайте в этом убедимся:

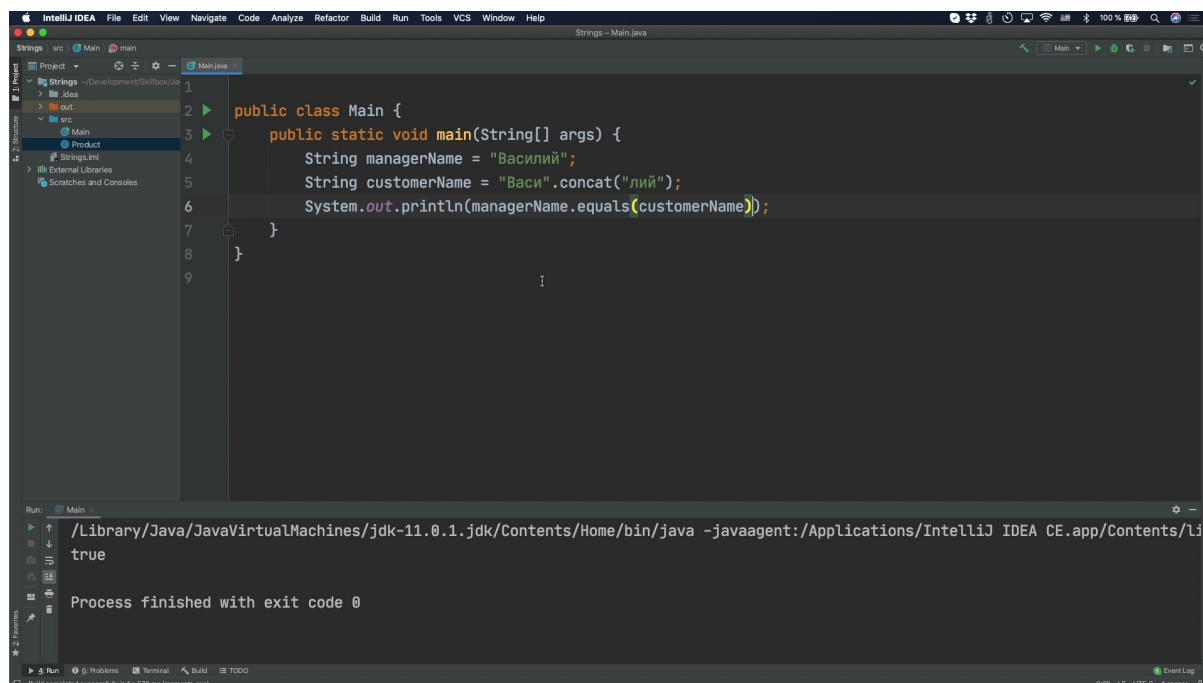


Если же сделать следующим образом, то, по сути, это по-прежнему будут две одинаковые строки, но сравнение с помощью двойного равно даст нам результат `false`:



Почему так происходит? Потому что двойное равно — это сравнение не объектов, а ссылок на них, и двойное равно не проверяет содержимое этих объектов. Даже если содержимое

двух разных объектов одинаковое, двойное равно выдаст результат false, поэтому для сравнения строк необходимо использовать не двойное равно, а метод equals, который как раз производит сравнение содержимого этих объектов. Вызываем его и видим, что в данном случае результат верный:



Поэтому при сравнении строк никогда не используйте двойное равно.

Возникает ещё один вопрос: почему же при сравнении таких объектов у нас выдаётся true, ведь это же, казалось бы, два разных объекта? Это происходит потому, что на самом деле обе переменные будут ссылаться на один и тот же объект в памяти. Это важная особенность работы строк в Java, о которой мы поговорим подробнее в одном из следующих видео этого модуля.

Поэтому, кроме всего прочего, вам необходимо запомнить, что для сравнения строк нужно пользоваться только методом equals и не использовать двойное равенство.

18:58–19:26 Итоги

Итак, вы познакомились со строками, узнали, как строки можно создавать и как проверять их длину. Также вы познакомились с понятием конкатенации, тремя основными способами конкатенации и их особенностями, разобрались, как правильно строки сравнивать.

В следующем материале мы подробно рассмотрим способы преобразования строк в числа и обратно.